



Planning

Given a complex customer scenario, assess Application Lifecycle Management maturity and identify the people, technology, and processes required.

Develop your career with **Salesforce Training and Certification Preparation**

Table of Contents

-  [Processes & Tools](#)
-  [Change Set Development Model](#)
-  [Org Development Model](#)
-  [Package Development Model](#)
-  [People & Roles](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the processes and associated tools that can be utilized to implement an ALM process.
-  Identify the key roles that are integral to an ALM process.
-  Identify the processes, tools, and roles necessary to implement an ALM process for a given customer scenario.



Introduction

Application Lifecycle Management (ALM) typically involves a specific **process** utilized for developing Salesforce applications. There are **three development** models available to implement an ALM process: the **Change Set Development Model**, the **Org Development Model**, and the **Package Development Model**.

Each model necessitates the use of certain features or tools. For instance, the Org Development Model is source-driven and employs a **source control system** for tracking changes, while the Package Development Model utilizes **unlocked packages** and **scratch orgs**.

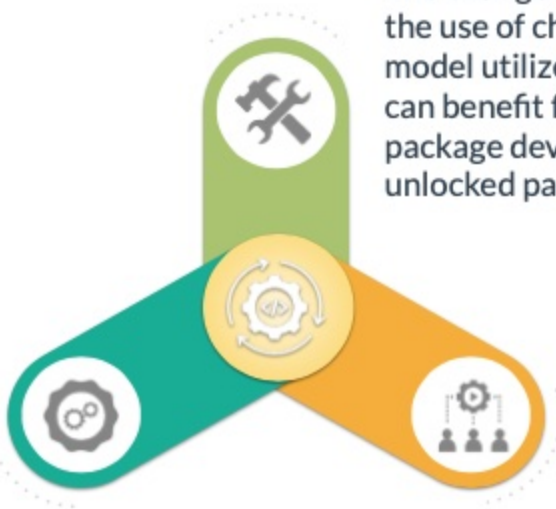
Various **key roles** are integral to the ALM process, including architects, developers, QA testers, project managers, release managers, and business analysts, among others. These roles play crucial functions in ensuring the success and efficiency of the application development lifecycle.



ALM Processes, Tools, and Roles

Processes

To implement an application lifecycle management process, three development models are available. These are change set development model, org development model, and package development model.



Tools

The change set development model focuses on the use of change sets. The org development model utilizes a source control repository and can benefit from a project management tool. The package development model involves the use of unlocked packages and scratch orgs.

Roles

Key people and roles in an application lifecycle management process include architect, developers, QA testers, project manager, release manager, business analyst, and stakeholders such as executive leadership and end users.

Processes & Tools

Table of Contents



Application Lifecycle Management Processes

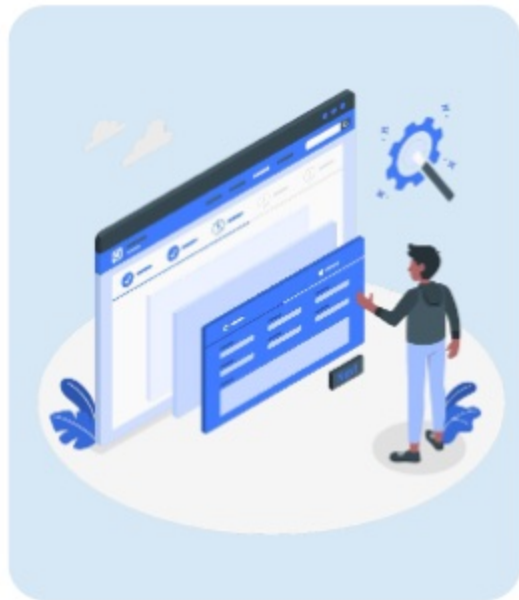
Three development models are available for managing the **lifecycle of Salesforce applications**.

❖ CHANGE SET DEVELOPMENT

In the Change Set Development Model, **change sets** are used to deploy **changes between related Salesforce environments**. Changes can be made in a non-production org, such as a developer sandbox, and then sent to another org for review and deployment.

❖ ORG DEVELOPMENT MODEL

In the Org Development Model, a **source control repository** is used to **track and manage changes**. Tools including Git, Salesforce CLI, and Salesforce Extensions for Visual Studio (VS) Code are utilized to migrate changes from one environment to another.

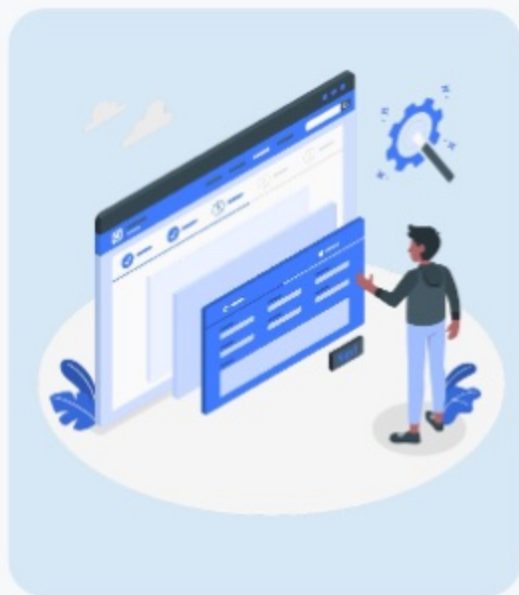


Application Lifecycle Management Processes

Three development models are available for managing the **lifecycle of Salesforce applications**.

❖ PACKAGE DEVELOPMENT MODEL

In the Package Development Model, modular units of functionality called **packages** are created. The **version control system** represents the **source of truth**. Developers use **scratch orgs** for the development of **unlocked packages**. **CI/CD tools** are typically used for continuous integration and deployment.



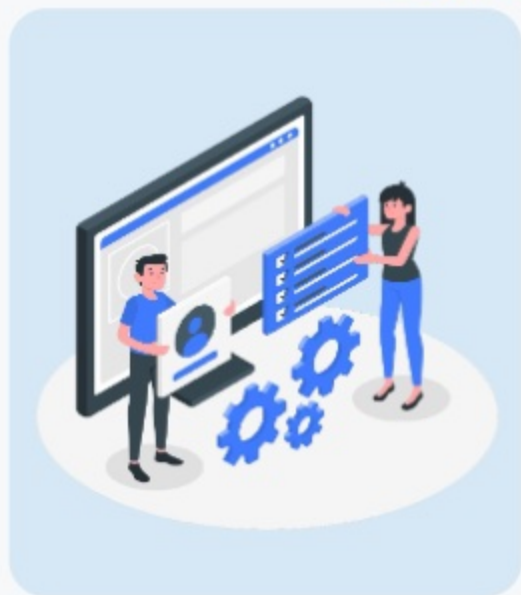
Change Set Development Model

[Table of Contents](#)



Change Set Development Model

The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.



◆ DEVELOP AND TEST

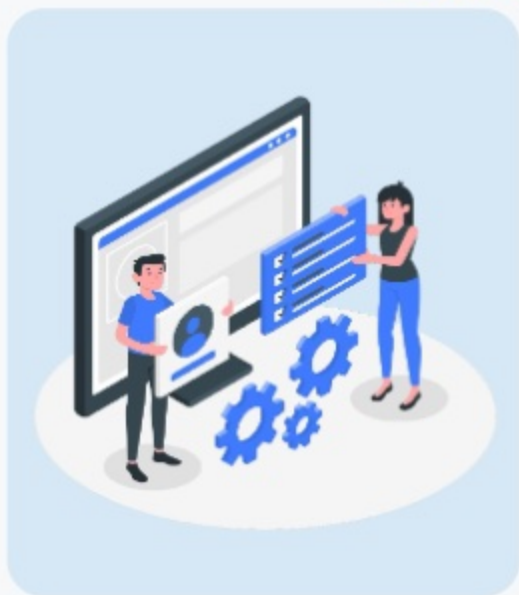
Developer sandboxes can be used to **create and test customizations**. A Developer sandbox contains **no production data** and can be used to keep changes **isolated** until they are ready to be integrated.

◆ BUILD RELEASE

After development and testing, customizations can be sent from each Developer sandbox to a shared **Developer Pro sandbox** for integration. Developer Pro sandboxes also do not contain production data.

Change Set Development Model

The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.



❖ TEST RELEASE

After integrating the changes, they can be sent to a **Full sandbox**, which is a **complete replica of production**. It can be used for **user-acceptance testing (UAT)**.

❖ RELEASE

After user-acceptance testing, the changes can be **validated** and **deployed** to the **production org** for final release. The **Full sandbox** can be used to **train users** without exposing production data.

Change Set Development Model

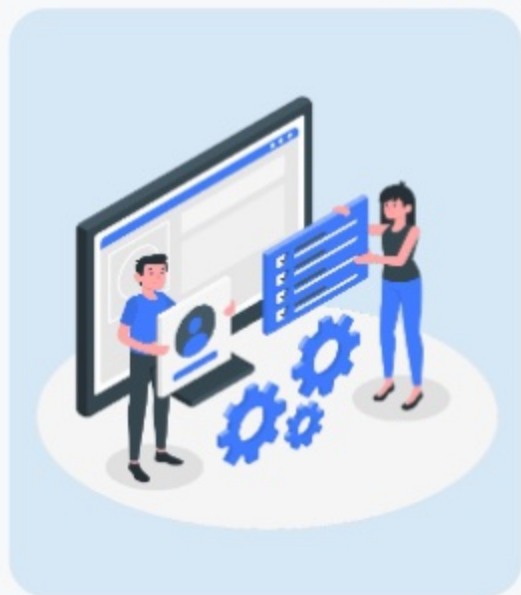
The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.

❖ MANUAL CHANGES

Changed metadata components that are **not supported** in Metadata API need to be **migrated manually**. The **Metadata Coverage report** can be accessed to view the supported metadata types.

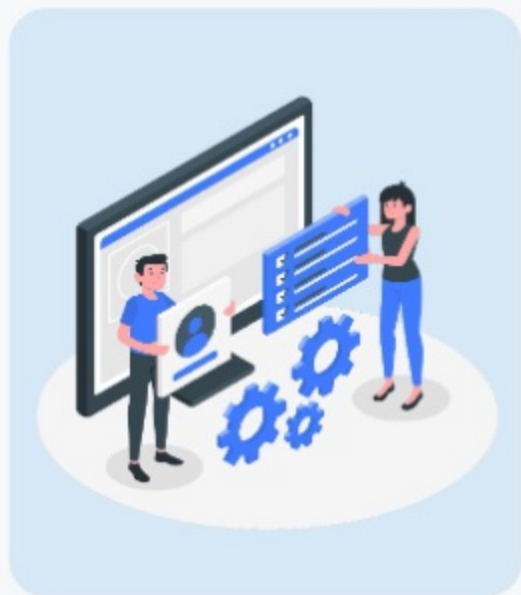
❖ CHANGE-TRACKING

To track changes, developers can use a **custom tool** to log each change they make. It can contain **fields for essential information** about each change in a release. The information can include **who** made the change, whether **manual migration** is required, the **component** impacted by the change, the **orgs** that have the change, and **when** the change was made.



Change Set Development Model

The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.



❖ DELETING AND RENAMING

Change sets **do not allow deleting or renaming** components. **Deletion** must be performed manually in the target org. A component can be renamed after first deleting it in the target org and then sending the new component in a change set.

❖ PROFILES AND PERMISSION SETS

Permission sets or profile settings can be used to specify permissions and other access settings in a change set. However, there are several related considerations.

Change Set Development Model

The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.

◆ OUTBOUND CHANGE SET

When changes are ready to be moved to another org, an outbound change set containing **new and changed metadata components** can be created in the **source org**. It must contain all the components necessary for the change, including **dependencies**.

◆ DEPLOYMENT CONNECTION

To allow a target org to receive inbound changes, a **deployment connection** needs to be **authorized** in the **target org**. This can be done on the **Deployment Settings** page in Setup by selecting **Allow Inbound Changes**.



Change Set Development Model

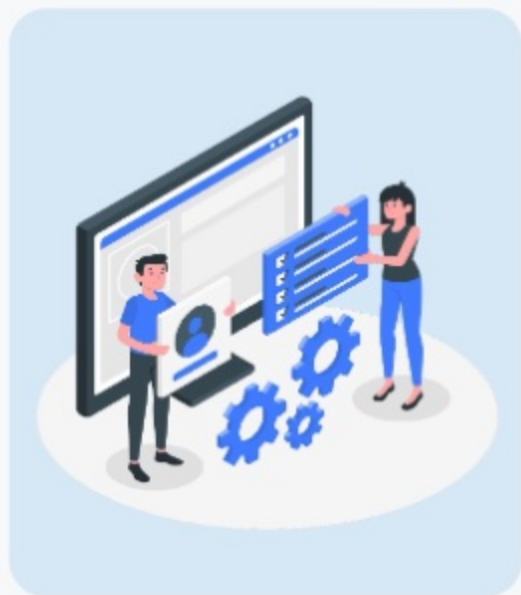
The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.

◆ UPLOAD

After authorizing the deployment connection, the **outbound change set** can be **uploaded** from the source org to the **target org**. For example, the upload can be performed in a Developer sandbox by selecting a Developer Pro sandbox as the target.

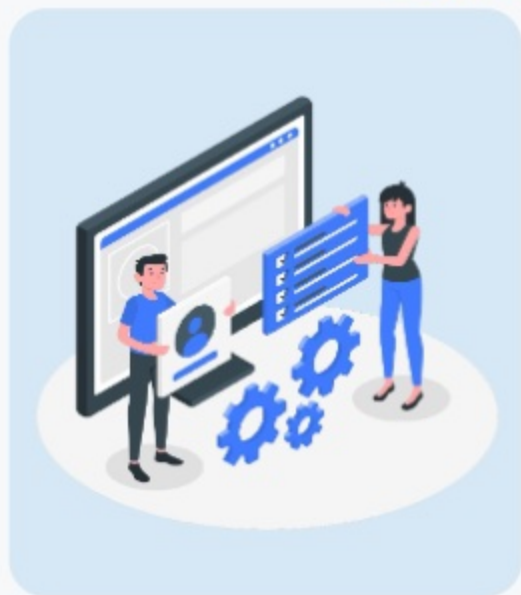
◆ INBOUND CHANGE SET

In the target org, the inbound change set uploaded in the source org can be **deployed**. If a deployment is **unsuccessful** for some reason, it is **rolled back**. If it **succeeds**, the **changes are committed** and the deployment **cannot be rolled back**.



Change Set Development Model

The Change Set Development model allows the use of **change sets** to send **customizations between Salesforce orgs** that are **affiliated with a production org**.



❖ VALIDATION

Before deployment to production, it is a best practice to **validate an inbound change set**, which shows the success or error messages that would occur during an actual deployment. After successful validation, the change set might qualify for a **quick deploy**.

❖ ROLLOUT

The change set is finally **deployed to a production org** at the scheduled rollout time. **Manual changes** are performed after the deployment.

Org Development Model

[Table of Contents](#)



Org Development Model

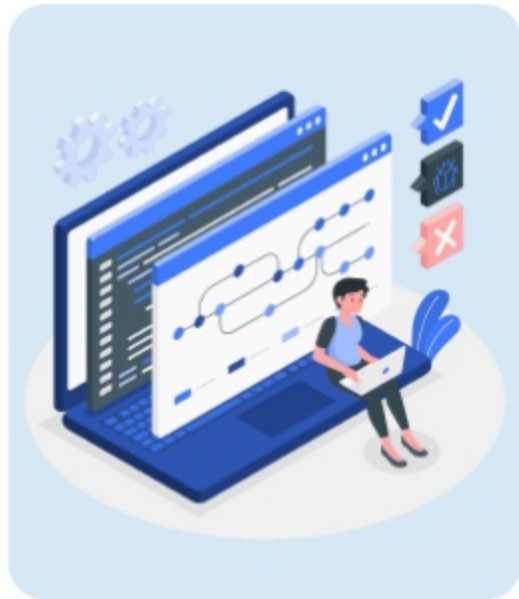
The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

❖ ENVIRONMENTS

The **Org Development Model** uses the same development and testing environments that are applicable to the Change Set Development Model. These include **Developer sandboxes** for development and testing, a **Developer Pro sandbox** for integration, and a **Full sandbox** for user-acceptance testing.

❖ SALESFORCE DX

A **Salesforce DX project** contains the **source and files** that represent the changes in an org. It is used to manage and collect the changes that need to be sent from one org to another. It contains a **configuration file named sfdx-project.json**.



Org Development Model

The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

❖ MANIFEST FILE

After changes have been tested, a **manifest file** named **package.xml** is created. It is a release artifact that lists all the components that need to be deployed to a target org.

❖ SOURCE CONTROL SYSTEM

A source control system is used to **merge and store all the changes**. It contains the **Salesforce DX project**. A popular example is **GitHub**.



Org Development Model

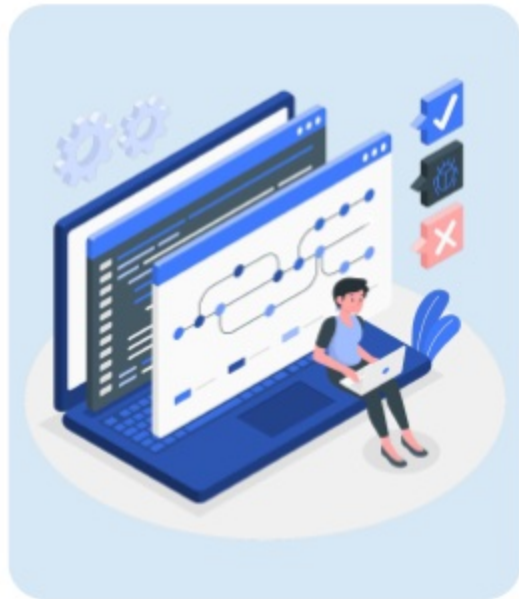
The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

◆ SALESFORCE CLI

Salesforce CLI is a **command-line interface** that can be used to **authorize** sandboxes, create and manage **DX projects**, import and export **test data**, retrieve and deploy **metadata**, and run and automate **tests**.

◆ SALESFORCE EXTENSIONS FOR VS CODE

Built on top of Salesforce CLI and VS (Visual Studio) Code, Salesforce Extensions for VS Code is an **integrated development environment** for custom development on the Lightning platform. It includes these VS Code extensions: **Salesforce CLI integration**, **Apex**, **Apex Replay Debugger**, **Aura Components**, and **Visualforce**

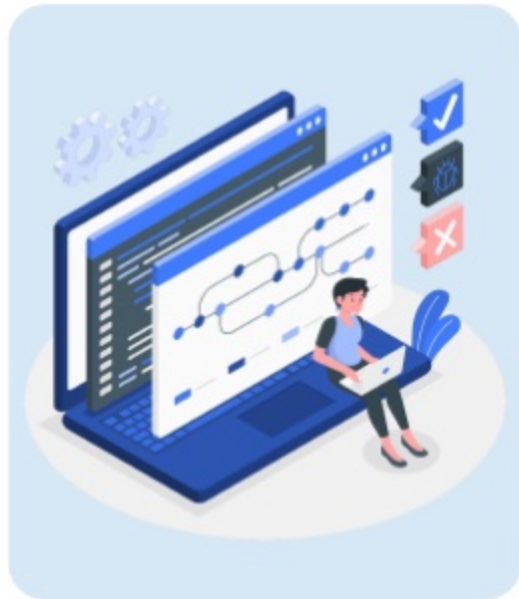


Org Development Model

The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

❖ CHANGE MANAGEMENT TOOLS

A developer can use a **change list (list, table, or spreadsheet)** to track the changes they made in their own dev org. They can use a **deployment run list** to list any manual changes required pre- and post-deployment. In addition, **project management tools** such as **Jira** and **Agile Accelerator** can be used to track requirements, feature work, and bugs. They can also be used to manage sprints and groom the backlog.



Org Development Model

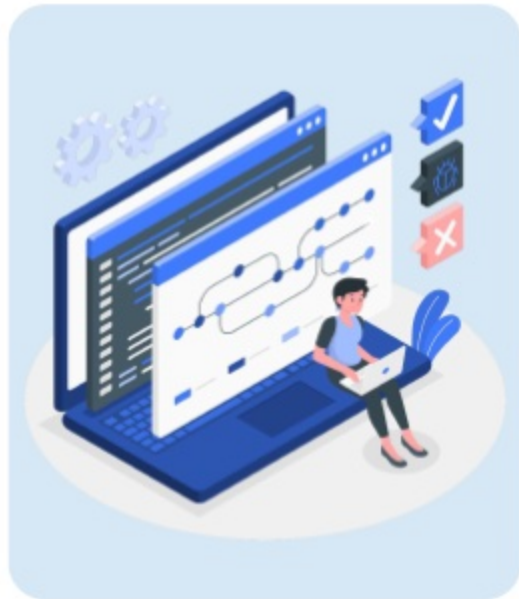
The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

◆ SOURCE CODE REPOSITORY

A **code repository** can be created in the **source control system** to organize the source files for a development project. A **Salesforce DX project** can be created and added to the repository. It can be committed and pushed to the default branch, such as **main**.

◆ DEVELOPMENT BRANCH

A developer can **clone the source code repository** and **create their own branch** for the changes they need to work on and deploy.



Org Development Model

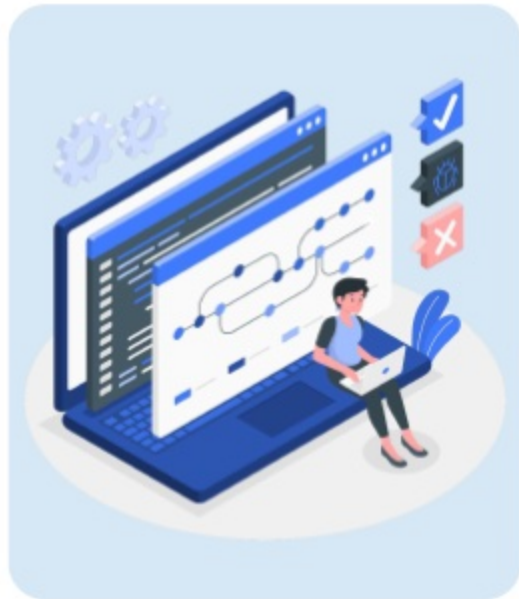
The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

❖ DEVELOP, RETRIEVE & DEPLOY

A developer can make changes in the **local DX project**. For example, they can create an Apex class for a new feature. Changes made in the Developer sandbox, such as a new custom object, can be **retrieved** to the local repository. Changes made in the local repository can be **deployed** to the Developer sandbox.

❖ COMMIT CHANGES

After all the changes have been made, they can be **committed** to the **source control repository**. A **pull request** can be created and **reviewers** can be assigned to it for **review** and **approval**. Once it is approved, the changes can be **merged** with the main branch.



Org Development Model

The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

❖ BUILD RELEASE ARTIFACT

After developing all the changes and committing them to the source control repository, the **final release artifact** (a manifest file named package.xml) can be built for all the components that need to be deployed to sandbox and production orgs.

❖ USER ACCEPTANCE TESTING

The release artifact can be deployed to a **partial sandbox** for **user-acceptance testing (UAT)**.



Org Development Model

The Org Development Model is a **source-driven development process** that utilizes a **source control repository** to store and track changes.

◆ STAGING

A **Full sandbox** can be used for **staging and regression testing**. Before deploying to production, **validation** can be performed, which **verifies the results of tests** and ensures that there's **no missing or invalid metadata**.

◆ RELEASE

Before releasing the changes to production, **pre-deployment tasks** are completed and a **validation** is performed. The validation sets up the **quick deploy**, which can be performed **within 10 days**. After the quick deploy, **post-deployment steps** are completed.



Package Development Model

Table of Contents



Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



◆ PACKAGE

In the **Package Development Model**, a release artifact called **package** is created. It is a **group of related code and customizations**. For example, a package can contain Lightning web components, Apex classes, custom objects, and custom fields for a custom application in an org.

◆ VERSION CONTROL SYSTEM

All the source and metadata for the packages is stored in a **source control repository**, which represents the **source of truth**.

Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



◆ SCRATCH ORGS

A developer can quickly spin up a **scratch (development) org** from the source to work specifically on a package. It can be configured with just the **edition, features, and preferences** needed for the package and used for **declarative development**.

◆ PACKAGE RELEASE

Teams can **develop separately** and build towards a release of the package. This allows **more frequent, independent releases**. The goal is to create packages that are **versionable, easy to maintain, update, install, and upgrade**.

Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



◆ UNLOCKED PACKAGES

Unlocked packages are suitable for internal business applications. An **unlocked package** can be used to organize **existing metadata**, package an **app**, **extend** an app, or package **new metadata**.

◆ SALESFORCE CLI

Salesforce CLI can be used to **manage** DX projects, create **scratch orgs**, **push** and **pull** metadata to and from a scratch org, and run **unit tests**. It can also be used as part of **build automation scripts** to **create and access** orgs, **deploy** source, **install packages**, and **run tests**.

Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



◆ SALESFORCE EXTENSIONS FOR VS CODE

Salesforce Extensions for VS Code is an integrated development environment that can be used to **create projects for package development** and **interact with the Salesforce CLI**.

◆ SANDBOXES

While scratch orgs are used for package development, sandboxes are used as targets/destinations for testing the installation of the package version. They can be used for **user acceptance testing**, **staging**, and **continuous delivery testing**.

Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



◆ SALESFORCE DX PROJECT

When a new Salesforce DX project is created, a **default 'package'** directory is created. The project directory stores the metadata in source format. Multiple packages can be organized into a single DX project.

◆ SYNCHRONIZATION

Salesforce DX **tracks local changes** as well as any **changes made in the scratch org**. Using Salesforce CLI, a **list of changes** can be **viewed**, local changes can be **pushed**, and changes in the scratch org can be **pulled**.

Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



❖ SYNCHRONIZATION

In the DX project format, large **source** files are broken down to make them **more digestible** and **easier to manage** with a version control system. The source structure makes it easy to find what needs to be updated. It also results in **fewer merge conflicts** during development.

❖ TESTING

To perform **manual or exploratory testing**, metadata is pushed into a separate scratch org. **Continuous Integration (CI)** is used to automate consistent test runs against every set of merged changes. **Package versions** are created and installed in each testing environment for **release testing** or **continuous delivery automation**.

Package Development Model

The Package Development model focuses on creating **modular units of functionality** called **packages**.



◆ CONTINUOUS DELIVERY

For continuous delivery, the process for deployment and release to production is tested. Testing is performed with the **package created in the build phase**. It is installed in a **sandbox** that is the **best representation of production**. The steps used to release to production are **replicated and tested** in the sandbox.

◆ DEPLOYMENT

After creating and testing the app or customizations, the **deployment artifact** is created. A **package version** is installed in the production environment.

People & Roles

[Table of Contents](#)



Key Roles in Application Lifecycle Management

In a typical application lifecycle management process for Salesforce development, various key **people and roles** are involved.



PROJECT MANAGER

A project manager is responsible for **planning, executing, and finalizing** projects according to strict **timeline** and **budget**.



RELEASE MANAGER

A release manager manages the **release process** and oversees the **development, testing, and deployment** phases of the release cycle.



ARCHITECT

A Salesforce architect is responsible for **designing, building, and delivering solutions** based on Salesforce best practices.

Key Roles in Application Lifecycle Management

In a typical application lifecycle management process for Salesforce development, various key **people and roles** are involved.



DESIGNER

A Salesforce designer is responsible for **designing the user interface** for an application or features while focusing on the **user experience**.



BUSINESS ANALYST

A business analyst **analyzes business needs** and **translates them into technical requirements**, ensuring that project goals align with business needs.



DEVELOPERS

Salesforce developers are responsible for **developing and customizing** features using **programmatic** approaches.

Key Roles in Application Lifecycle Management

In a typical application lifecycle management process for Salesforce development, various key **people and roles** are involved.



ADMINISTRATORS

Salesforce administrators are responsible for **managing the Salesforce environments** and making **declarative changes**.



QA TESTERS

Quality Assurance (QA) testers ensure that the developed application **functions as intended** and **identify bugs or issues**.



STAKEHOLDERS

Stakeholders provide **crucial feedback and input**, particularly during the planning and testing phases.

Stakeholder Groups

There are various stakeholder groups who participate in the **application lifecycle management process**.

EXECUTIVE LEADERSHIP

This group includes **executive sponsors** and the **steering committee**. They **communicate key information** to leaders and end users and **receive recurring updates** on the readiness status.



MANAGERS & SUPER USERS

Managers **build awareness and understanding** and are responsible for **championing Salesforce readiness activities to their staff**.



LEADERSHIP TEAM

Leaders can be from departments such as sales, service, and marketing. They act as **visible project sponsors** for Salesforce implementation and associated changes.

CORE END USERS

Core end users **provide key input** on the needs of the primary users based on received **communications** and **attend training and readiness activities**.

Business Scenario

Table of Contents



Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Technology Group, a multinational corporation, specializes in providing complex Salesforce solutions to a diverse range of clients. The company's portfolio includes large-scale Salesforce integrations, custom application development, and system overhauls.

Two regional teams provide Salesforce development services. The EMEA team focuses on clients in Europe, and the APAC team serves the Asia-Pacific market, but they often work together on some projects. Given the complexity and scale of their projects, the company faces challenges in maintaining consistency, quality, and efficiency across the teams.

A Salesforce architect is brought on board to assess the maturity of their current ALM processes, evaluate the existing people, technology, and processes utilized by the two regional teams, and recommend improvements.

Requirements & Solutions

Table of Contents



Requirement 1



SCENARIO

The two regional teams need to work together on a large-scale Salesforce implementation project for a multinational banking corporation. This project involves initial system design, integration with legacy banking systems, and implementation of advanced security protocols. Given the project's complexity, there's a need for a **sophisticated application lifecycle management process** that supports **rigorous testing and quality control** and aligns with the organization's high maturity level. This process must involve a **tool** to manage requirements and defects and facilitate seamless communication and collaboration.



Solution 1 (Part 1)



SOLUTION

For the large-scale Salesforce implementation project, the teams should adopt the **org development model**. A **source control system** like Git should be integrated with the development lifecycle for effective code management and version control. The developers of the two regional teams should work within separate Salesforce **Developer sandbox** environments, which would promote focused development while maintaining project alignment. Developed features can be integrated in a **Developer Pro sandbox**. A **Partial sandbox** can be used for user-acceptance testing. A **Full sandbox** can be used for staging and training.



Solution 1 (Part 2)

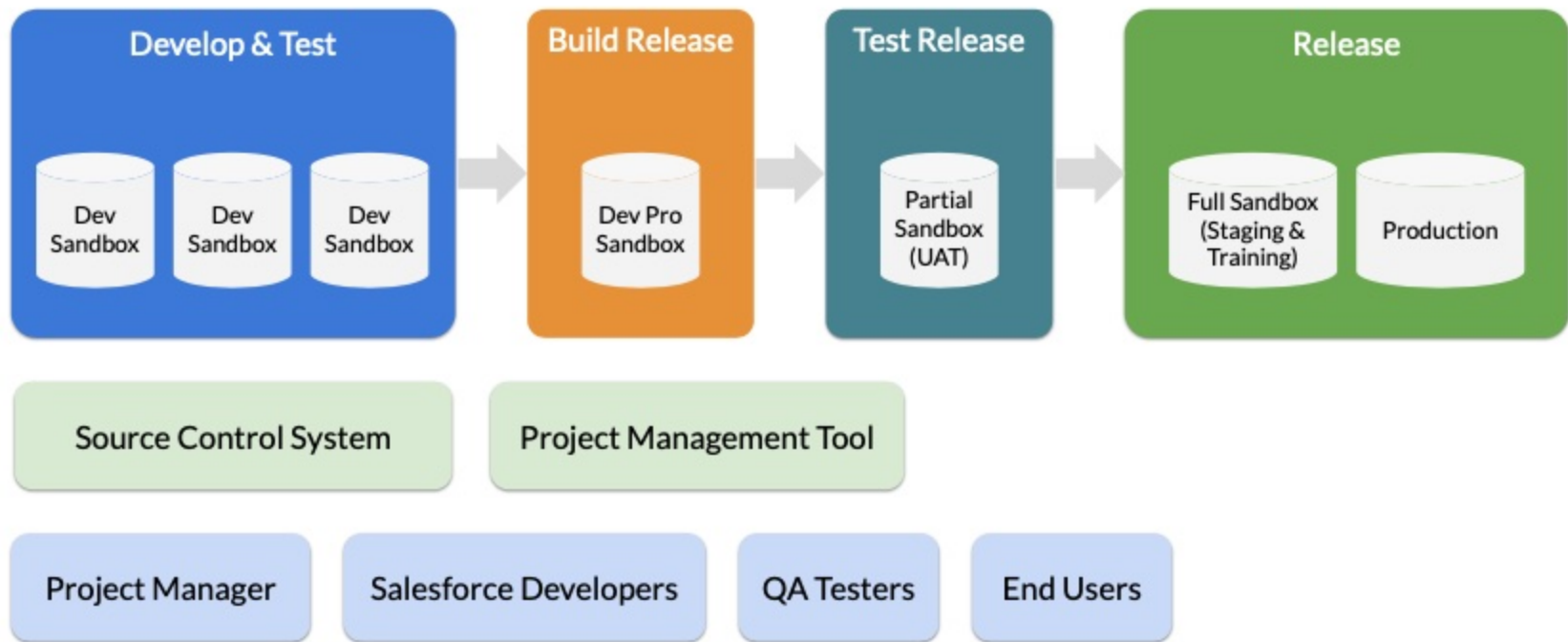


SOLUTION

A **project management tool**, such as Jira or Agile Accelerator, should be implemented to manage requirements, track development tasks, and log defects across teams. Regular cross-regional coordination meetings should be scheduled to ensure synchronicity in development efforts and address any emerging issues.



Solution 1 (Part 3)



Requirement 2 (Part 1)



SCENARIO

The EMEA team needs to work on a complex Salesforce project. It involves the development of a series of **complex, interrelated applications** for a financial services client. These include a custom financial tracking tool, a human capital management application, and data analytics capabilities. Given the sophistication of these applications and the client's expectation for **seamless integration and upgradability**, a **high degree of coordination and control** is required throughout the development lifecycle.



Requirement 2 (Part 2)



SCENARIO

The requirement is for an Application Lifecycle Management (ALM) process that aligns with the company's high maturity level, leveraging the **package development model** for modular development, **scratch orgs** for isolated testing environments, and a **source control system** for versioning, along with the appropriate **tools and roles** to support these processes.

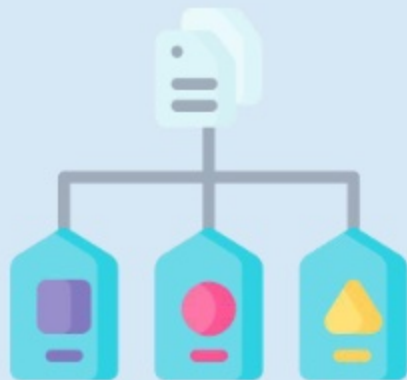


Solution 2 (Part 1)



SOLUTION

The project should be broken down into smaller, manageable packages using the package development model. Each module (financial tool, HCM app, and analytics) should be developed as an independent **unlocked package**, allowing for **easier maintenance** and **upgradability**. A **source control system** like Git, integrated with **Salesforce DX**, should be used for version control. Each development stream should have its own branch, ensuring clean separation of code and configurations. **Scratch orgs** should be utilized by individual developers and testers. They can provide isolated environments to develop and test each module without impacting other development streams.

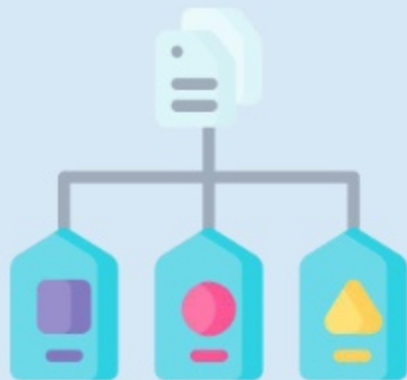


Solution 2 (Part 2)

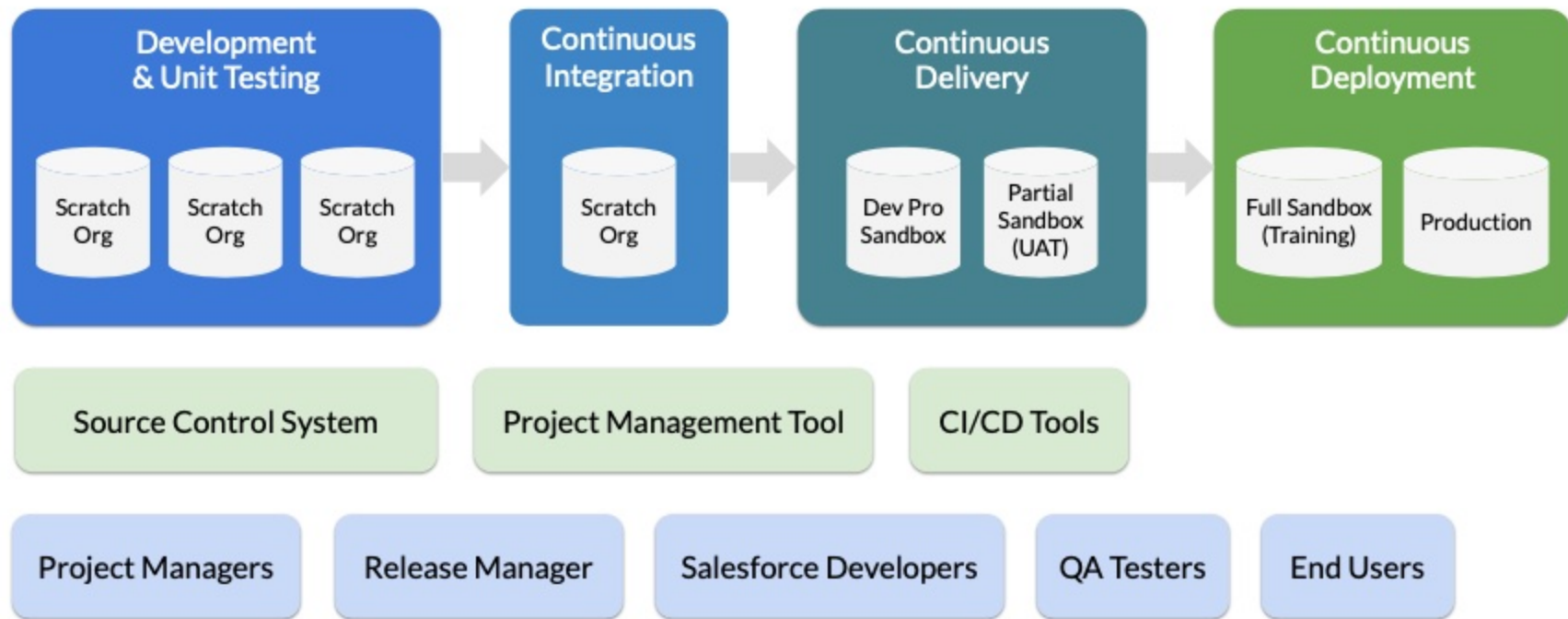


SOLUTION

The ALM process should consist of various key roles and people. **Salesforce Developers** should be assigned to coding and configuration tasks within their respective packages. **QA Testers** can use scratch orgs for testing each package, ensuring functionality and interoperability. **Project Managers** can oversee the project, track progress, and coordinate using an ALM tool like Jira or Agile Accelerator. In addition, **CI/CD tools** can be utilized to automate the build and deployment process for efficiency and reduced error rate. A **Release Manager** can assist in creating a unified plan for releases and also identify and assure use of technical deployment tooling (such as CI/CD tools).



Solution 2 (Part 3)



Learn More

-  [Application Lifecycle and Development Models](#)
-  [Change Set Development Model](#)
-  [Org Development Model](#)
-  [Package Development Model](#)
-  [Stakeholder Management Structure](#)
-  [Managing Governance with a Center Of Excellence](#)

